

Universidad Autónoma de Madrid

Escuela Politécnica Superior



Grado en Ingeniería Informática

TRABAJO DE FIN DE GRADO

TODO: TÍTULO DEL TRABAJO DE FIN DE GRADO

Javier López Cano
Tutor: Ramón Díaz Uriarte

Junio 2021

TODO: TÍTULO DEL TRABAJO DE FIN DE GRADO

Autor: Javier López Cano
Tutor: Ramón Díaz Uriarte

Dpto. de Bioquímica
Escuela Politécnica Superior
Universidad Autónoma de Madrid

Junio 2021

Agradecimientos

TODO: Agradecimientos.

“TODO: Cita relevante” TODO: Autor de la cita

Abstract

Abstract — TODO: Resumen en inglés, 250-500 palabras.

Key words — TODO: Palabras clave en inglés, separadas por coma.

Resumen

Resumen — TODO: Resumen en español, 250-500 palabras.

Palabras clave — TODO: Palabras clave en español, separadas por coma.

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	1
1.3. Estructura del documento	2
2. Estado del Arte	3
3. Diseño	5
3.1. Introducción. Terapia adaptativa y variables de usuario	5
3.2. Definición de variables y reglas	6
3.3. Algoritmo de actualización	7
3.4. Definición de la estructura de salida	8
4. Desarrollo	9
4.1. Introducción	9
4.2. Unificación de Ramas	10
4.3. Optimización del código de intervenciones	11
4.4. Implementación de variables de usuario y reglas	12
4.4.1. En R	12
4.4.2. En C++	14
4.5. Integración en el flujo principal de código	18
4.6. Variables de usuario en las intervenciones. Terapia adaptativa	20
5. Pruebas y Resultados	21
5.1. Comprobación de funcionalidad previa	21
5.2. Tests de variables de usuario	22
5.3. Tests de intervenciones	23
5.4. Resultados	23
6. Conclusiones y trabajo futuro	25
6.1. Conclusiones	25
6.2. Líneas de trabajo futuro	25
Apéndices	27

A. Ejemplos de bloques y comandos útiles en LaTeX **29**
A.1. Ejemplo de sección 29

Índice de tablas

Índice de figuras

A.1. Logo de la Universidad Autónoma de madrid.	30
---	----

1

Introducción

1.1. Motivación

- ¿Intro sobre el cáncer e intervenciones?
- ¿Terapia adaptativa?

1.2. Objetivos

- Familiarizarse con OncoSimulR (especialmente el flujo general de simulación y las intervenciones)
- Aprender sobre R y C++
- Completar el merge de las ramas anteriores solucionando todos los errores que surjan
- ¿Optimizar el código de intervenciones?
- Añadir la posibilidad de definir variables de usuario arbitrariamente complejas tanto como salida en sí mismas como para emplearse en las intervenciones
- Generar una batería de pruebas para comprobar el correcto funcionamiento de las variables de usuario y las intervenciones que las emplean

1.3. Estructura del documento

TODO: Descripción de la estructura del documento

2

Estado del Arte

3

Diseño

3.1. Introducción. Terapia adaptativa y variables de usuario

Como hemos visto en la sección 1.1 (TODO), la base fundamental de la terapia adaptativa es la posibilidad de crear tratamientos específicos para determinados momentos en el desarrollo de un tumor. Dentro del programa **OncoSimulR** los tratamientos están simulados a través de las **intervenciones**. Por tanto, para simular la terapia adaptativa debemos lograr que estas intervenciones puedan depender del estado de desarrollo del tumor, es decir, del estado de la simulación en el momento concreto en que se va a ejecutar la intervención.

Para ello se ha decidido añadir al programa la posibilidad de definir variables de usuario, es decir, se ha incluido la posibilidad de que el usuario defina una serie de variables cuyo valor sea una función arbitrariamente compleja de distintos parámetros de la simulación. A lo largo de la simulación tumoral, estas variables se irán actualizando de acuerdo a la fórmula introducida por el usuario, haciendo que los valores vayan variando en función de la evolución del tumor.

Estas variables podrán luego ser empleadas para elaborar las expresiones que definen la acción a realizar en una intervención (como se ha visto en el punto 2 TODO), permitiendo así condicionar el efecto de una intervención al estado del tumor en el momento de la ejecución de esta, es decir, permitiendo la terapia adaptativa.

Con esto en mente procedemos al diseño de estas variables de usuario.

3.2. Definición de variables y reglas

A la hora de plantearse el diseño necesario para esta funcionalidad, lo primero que necesitamos saber es qué son y como se definen las variables de usuario que vamos a implementar.

Las variables de usuario son variables arbitrariamente complejas que un usuario puede definir a partir de combinaciones o funciones de algunas variables ya existentes en el programa. Para obtener estas variables arbitrariamente complejas vamos a necesitar de una serie de reglas que determinen el valor de estas variables en cada momento de acuerdo a las condiciones propuestas por el usuario para cada una de las variables.

Por tanto podemos determinar que por un lado tendremos un listado de variables, cada una de las cuales debera contener:

- **Nombre:** nombre de la variable de usuario que permite identificarla y diferenciarla de el resto de variables definidas.
- **Valor:** valor numérico de la variable en cada momento.

Por otro lado tendremos un listado con las reglas que permiten modificar y actualizar las distintas variables definidas. Estas reglas cuentan de:

- **ID:** identificador único de cada regla que permite diferenciarla del resto de reglas definidas.
- **Condition:** la condición que, si se cumple, hará que la variable de usuario se modifique.
- **Action:** acción que se llevará a cabo en caso de que la **Condition** se cumpla, es decir, es el conjunto de modificaciones a realizar a las variables de usuario en caso de que se cumpla la condición de la regla. podrá contener una o varias fórmulas que determinarán los nuevos valores de una o varias variables de usuario.

En este punto debemos mencionar también, qué variables del programa se ha decidido que sean empleables para la modificación de variables y definición de las reglas por parte del usuario y los motivos por los que se ha decidido esto. Las variables disponibles para la definición de las reglas son:

- **birthRate:** la tasa de nacimiento correspondiente a lcada genotipo, es decir,es un número que determina la probabilidad que tiene una célula con este genotipo de reproducirse, y por tanto aumentar su número total en 1 en un ciclo de tiempo.
- **deathRate:** la tasa de muerte correspondiente a cada genotipo, es decir, es un número que determina la probabilidad que tiene una célula con este genotipo de morir, y por tanto reducir su número total en 1 en un ciclo de tiempo.

- **mutationRate**: la tasa de mutación correspondiente a cada genotipo, es decir, es un número que determina la probabilidad que tiene una célula con este genotipo de mutar, y por tanto variar alguno de sus genes, pasando de ser de este genotipo a ser de otro genotipo distinto en 1 en un ciclo de tiempo.
- **N**: número total de células en la simulación, teniendo en cuenta todos los genotipos existentes.
- **n_x**: número de células de un determinado genotipo concreto, por ejemplo, **n_A** será el número de células del genotipo A, y **n_A_B** las del genotipo A_B
- **T**: tiempo transcurrido desde el inicio de la simulación, en segundos, con precisión de milisegundos.
- **Variables de usuario**: otras variables de usuario definidas en la misma simulación.

Estas variables se han añadido con la finalidad de dar a las variables de usuario la mayor flexibilidad posible para que el usuario las defina de acuerdo a sus necesidades, considerando todas las variables interesantes tanto para la terapia adaptativa como desde un punto de vista biológico para estudiar su evolución.

Las variables **N**, **n_x** y **T** se consideran necesarias para la terapia adaptativa, ya que permiten definir variables de usuario que se modifiquen de acuerdo a la evolución del tamaño del tumor (**N**), a la cantidad de las células de un determinado tipo, que se consideren más perjudiciales o sensibles a un determinado tratamiento (**n_x**) o al tiempo transcurrido (**T**). Estas variables definidas de acuerdo a estos atributos se podrían emplear luego para definir las intervenciones que permite el programa OncoSimulR, permitiendo adaptarlas de forma adecuada a las distintas combinaciones posibles de estos 3 atributos a lo largo de la simulación, generando así la posibilidad de simular terapia adaptativa. A estas 3 variables se les suman las propias variables de usuario, para poder así generar combinaciones aún más complejas que permitan simular escenarios altamente enrevesados.

Las otras variables **birthRate**, **deathRate** y **mutationRate**, se han añadido con una finalidad más informativa, ya que, a pesar de no ser tan interesantes desde el punto de vista de la terapia adaptativa (aunque pueden aplicarse en algunos casos), dan una información importante acerca de la adaptabilidad de los distintos genotipos a lo largo de la simulación, haciendo que el estudio de la evolución de variables de usuario que sean combinación de estas variables a lo largo de la simulación pueda tener un gran interés para el estudio tumoral.

3.3. Algoritmo de actualización

El algoritmo de actualización de las variables de usuario es a priori, bastante sencillo, pero su principal complejidad radica en su implementación e integración en el flujo de código principal, como veremos en la sección 4.

En rasgos generales el algoritmo de actualización es:

1. El usuario define las varibales de usuario que desea y las reglas para modificarlas
2. Se verifica que estan definidas correctamente
3. Cada unidad de tiempo muestreada en la simulación:
 - a) Para cada regla:
 - 1) Si se cumple la Condition:
 - a' Modificamos el valor de las variables de usuario según lo especificado por el usuario en el atributo Action
 - 2) Se pasa a la siguiente regla volviendo a 1)
 - b) Se procede a la ejecución de las intervenciones con los valores de las variables calculados

3.4. Definición de la estructura de salida

Para obtener los datos de evolución de las variables de usuario definidas en el tiempo a lo largo de la simulación, se deberán modificar la salida del programa de simulación para que incluya estos. Se debe añadir por tanto a la salida

- Una lista con los nombres (“ids”) de las variables de usuario creadas.
- Una matriz con los valores de las variables de usuario en cada unidad de tiempo muestreada (en el mismo orden que los nombres de las variables en en la lista de nombres de las variables de usuario).

La finalidad de añadir estos datos a la salida de la simulación es poder emplear las variables de usuario (que son arbitrariamente complejas) como datos de salida y no solo como valores a emplear en otros procesos como las intervenciones. Por ejemplo, obtener estos datos permite al usuario graficar la evolución en el tiempo de algunas variables de usuario creadas, lo que puede ser de gran interés para evaluar la evolución del tumor en la simulación. Además, la adición de estos datos en la salida facilitará enormemente las labores de testeo de la funcionalidad implementada.

4

Desarrollo

4.1. Introducción

En esta sección se va a explicar en primer lugar cómo se ha procedido con la unificación de las dos ramas principales con funcionalidad divergente de las que se partía, incluyendo la solución de los errores derivados del merge realizado, asíni como de errores previos de funcionalidad independientes del merge. Tras esto se procederá a la explicación de la implementación realizada para la funcionalidad definida en el apartado de diseño.

El desarrollo de los ficheros de código que componen esta funcionalidad se ha realizado en los lenguajes R y C++ por ser los lenguajes en los que ya está desarrollado el grueso del programa **OncoSimulR**. R se emplea para la interacción con el usuario, por ser un lenguaje muy común en bioinformática; por su parte C++ está presente para la ejecución del algoritmo de simulación, por er un lenguaje más eficiente para la ejecución de algoritmos complejos, reduciendo significativamente el coste temporal de ejecución de la simulación.

A lo largo de este apartado se mencionará reitradamente el uso de las librerías **Rcpp** y **ExprTk**.

- **Rcpp** es paquete del lenguaje R que permite el paso de datos y objetos entre los lenguajes R y C++, transformando los tipos de dato de uno de los lenguajes a sus equivalentes más próximos en el otro. De este modo, este paquete permite la comunicación fluida entre ambas partes del programa, la parte dedicada a la comunicación con el usuario (desarrollada en R) y la parte computacionalmente más constosa que implementa el grueso del algoritmo de simulación tumoral (desarrollada en C++).

- **ExprTk** es una librería definida en C++ que permite el parseo de expresiones matemáticas complejas. Esta librería ya estaba siendo empleada en el programa para el parseo de las expresiones que definen las intervenciones, pero en esta nueva funcionalidad va a ser empleada también para parsear las expresiones que permitirán la modificación de las variables de usuario en tiempo de simulación.

4.2. Unificación de Ramas

Cómo se ha explicado en el punto 2 (TODO) al iniciar este trabajo se partía de 2 ramas con versiones distintas del programa OncoSimulR cada una con una funcionalidad recientemente implementada en cada una de ellas. Estas 2 ramas debían unificarse para proceder a la implementación de la nueva funcionalidad sobre la versión combinada de ambas.

La unificación de ambas ramas se realizó de manera manual, ya que el merge automático proporcionado por la herramienta Git generaba un gran número de conflictos y no respetaba la funcionalidad de las ramas. Al proceder, se decidió realizar el merge sobre la rama que contenía la funcionalidad del birth y death rate (explicada en el punto 2 TODO) por ser la que más modificaciones introducía sobre el código base de la simulación, y sobre esta se introdujeron los cambios correspondientes a la rama con la funcionalidad de las intervenciones (explicada en el punto 2 TODO).

Para realizar la unificación de ambas ramas se procedió del siguiente modo:

1. Sobre la rama que se ha indicado anteriormente se añadieron los ficheros de código R y C++ exclusivos de la funcionalidad de intervenciones (intervention.cpp, intervention.h, intervention.R, multivariorant_hyergeometric.cpp y multivariant_hyergeometric.h), los ficheros de testeo de esta funcionalidad (test.intervention.R y test.Z-intervention.R) y los ficheros de documentación y viñetas que explican e ilustran el funcionamiento de esta.
2. Tras esto se procedió a la modificación de la función **updatePopulations** del fichero intervention.cpp, encargada de actualizar las poblaciones de los distintos genotipos tras la ejecución de una intervención, para que emplease el birth y death rate implementado en la funcionalidad de la otra rama.
3. Se actualizó el flujo de código principal del algoritmo BNB de Mahler (TODO: explicado en el punto 2) en el fichero BNB_nr.cpp para añadir las llamadas necesarias para añadir la funcionalidad correspondiente a la ejecución de las intervenciones.
4. Por último se modificaron las cabeceras y llamadas a todas las funciones del programa OncoSimulR para que tomaran de forma adecuada el argumento interventions, teniendo especial cuidado al tratar con los ficheros RcppExports.R, RcppExports.cpp y OncoSimulR_init.c, encargados de la comunicación entre los ficheros en R y en C++ mediante la librería Rcpp.

Posteriormente se realizó un merge automático con una tercera rama modificada por el tutor de este trabajo para incluir unas pequeñas modificaciones en la documentación del programa.

Al finalizar este proceso, se procedió a la comprobación de que la funcionalidad de ambas ramas era correcta en la rama combinada mediante la ejecución de los tests pertinentes, y a asegurar que el merge no ha afectado al resto de características del programa. Todo este testeo se explicará de forma más detallada en el punto 5.1 TODO.

4.3. Optimización del código de intervenciones

Durante la realización del merge de las ramas previas, se encontraron algunos problemas de optimización en el código de las intervenciones que hacían que el tiempo de ejecución fuese más alto de lo deseado, por tanto antes de proseguir con la nueva funcionalidad se procedió a solucionar esto.

El principal problema detectado en este aspecto eran fragments de código que se ejecutaban en todas las iteraciones, independientemente de si se realizaban intervenciones o no, cuando solo era necesaria su ejecución cuando estas si se realizaban. Si bien estos fragmentos de código no eran en sí mismos muy costosos coimputacionalmente, en una simulación de gran tamaño, con un gran número de iteraciones (como las que se realizan habitualmente en est programa), estos fragmentos se estarían ejecutando de forma innecesaria un enorme número de veces, haciendo que la variación en el tiempo de ejecución sí fuese bastante significativo.

Los cambios realizados, por tanto, fueron:

- Modificación de la función **executeInterventions** de interventions.cpp:
 - Cambiamos la función para que devuelva true si ha habido una intervención y false en caso contrario en lugar de devolver void como hasta ahora. Esto nos permite desde el algoritmo principal saber si ha habido o no intervención para ejecutar los fragmentos de código necesarios, además nos permite desde fuera obtener los tiempos en que se ejecutan las intervenciones, facilitándonos el testeo posteriormente.
 - Hacemos que en la función solamente se llame a **updatePopulations** (que actualiza las poblaciones de cada genotipo) en los casos en los que se ha ejecutado alguna intervención, puesto que son los únicos casos en que las poblaciones pueden haber cambiado.
- Modificamos también la función **parseWhatHappens** para que devuelva void en vez de true/false. Antes devolvía true si se ejecutaba sin errores y en caso de error lanzaba una excepción y devolvía false, lo cual no tenía sentido, por tanto ahora no devuelve nada y en caso de error simplemente lanza una excepción.

Tras esto de nuevo procedemos a comprobar que la funcionalidad es correcta mediante los tests explicados en el punto 5 TODO, y procedemos a comenzar con la nueva funcionalidad.

4.4. Implementación de variables de usuario y reglas

4.4.1. En R

Desde R debemos hacer posible que el usuario pueda definir las variables de usuario que desee, y las reglas que quiere que se emplee para su cálculo y actualización a lo largo de la simulación, así como permitir que pase estas como argumento a la función **OncoSimulIndiv** de OncoSimulR, que es la función responsable de llevar a cabo la simulación.

Para ello en primer lugar vemos como se deben definir las variables de usuario y las reglas en R:

- **Variables de usuario:** las variables de usuario hemos visto en el punto 3.2 que simplemente contienen un nombre que identifica a la variable, y un valor numérico. Por tanto el usuario definirá en R una simple lista con todas las variables de usuario que desee crear para la simulación con estos atributos, del siguiente modo:

```
userVars <- list(  
  list(Name = "user_var_1",  
        Value = 0  
  ),  
  list(Name = "user_var_2",  
        Value = 0  
  ),  
  list(Name = "user_var_3",  
        Value = 0  
  )  
)
```

- Name: string que contiene el nombre que identificará a la variable, debe ser único para la variable.
 - Value: valor inicial que tomará la variable al inicio de la simulación.
- **Reglas:** del mismo modo hemos visto en el 3.2 que las reglas constan de un ID, un Condition y una Action. De igual forma el usuario definirá estas reglas mediante una lista en R de la forma:

```
rules <- list(  
  list(ID = "rule_1",  
        Condition = "T > 20",
```

```

        Action = "user_var_1 = 1"
    ), list(ID = "rule_2",
        Condition = "T > 30",
        Action = "user_var_2 = 2"
    ), list(ID = "rule_3",
        Condition = "T > 40",
        Action = "user_var_3 = 3; user_var_2 = 1"
    )
)

```

- ID: string identifica la regla y la distingue del resto, debe ser único para cada regla.
- Condition: expresión con valor true/false que determina si se ejecuta o no la regla. Puede emplear las variables especificadas en el punto 3.2.
- Action: una o varias expresiones con valor numérico separadas por “;” que determinan el valor que tomarán las variables de usuario en caso de que se ejecute la regla. Del mismo modo que el campo Condition, puede emplear las variables especificadas en el punto 3.2.

Partiendo de estas definiciones, creamos en R una serie de funciones que comprueban que el usuario ha especificado las variables de usuario y reglas de forma correcta y las adaptan para que puedan pasarse a C++ de forma correcta mediante la librería Rcpp. Estas funciones son en el caso de las variables de usuario:

- **check_same_name**: comprueba que en la lista definida no existen varias variables de usuario con el mismo nombre.
- **verify_user_vars**: comprueba que todas las variables de usuario definidas en la lista tienen todos los atributos necesarios (Name y Value).
- **createUserVars**: mediante llamadas a las 2 funciones anteriores comprueba que la lista de variables de usuario definida es correcta, y si lo es, la devuelve.

Y en el caso de las reglas:

- **check_double_rule_id**: comprueba que en la lista de reglas definida no hay 2 reglas con el mismo id.
- **check_action**: comprueba la correcta especificación de la expresión del campo Action, para ello comprueba que, si hay varias expresiones, estas están correctamente separadas por “;” y que cada una de las expresiones tiene la forma: <variable de usuario>= <expresión>.
- **verify_rules**: comprueba que las reglas contienen todos los atributos necesarios (id, Action y Condition), y que estos son correctos llamando a las 2 funciones anteriores.

- **transform_rule**: modifica las expresiones Action y Condition de las reglas, ya que aunque en estas las variables **n_X** (explicadas en 3.2) se especifican como **n_A**, **n_B**, etc; en C++ los genotipos se definen con números y no con letras, por tanto esta función transforma **n_A**, **n_B**... en **n_1**, **n_2**...
- **adapt_rules_to_cpp**: mediante llamadas a la función anterior, modifica las expresiones action y condition de las distintas reglas definidas en la lista para adaptarlas a C++.
- **createRules**: mediante llamadas a **verify_rules** y a **adapt_rules_to_cpp** comprueba que las reglas definidas son correctas y las adapta a C++ devolviendo la lista de reglas ya transformada.

Por último, en R, se han modificado las llamadas y la cabecera de la función `OncoSimulIndiv` y de las demás funciones internas de `OncoSimulR` para que reciban de forma adecuada los nuevos argumentos `userVars` (lista de variables de usuario) y `rules` (listado de reglas).

4.4.2. En C++

Ya en C++ procedemos a desarrollar en TAD que maneja las variables de usuario y reglas definidas por el usuario en R. Para ello en primer lugar definimos las estructuras que contienen estos objetos. Para las reglas creamos la estructura **Rule** en C++, del siguiente modo:

```
typedef struct {  
    std::string id; // identifier of the rule  
    std::string condition; // condition that must be true for the  
        ↪ rule to execute  
    std::string action; // action to be performed when the condition  
        ↪ is satisfied  
} Rule;
```

Donde todos los campos son strings que corresponden exactamente a los atributos especificados en R por el usuario, explicados en el punto anterior 4.4.1.

Por otro lado definimos otra estructura en C++ llamada **UserVarsInfo**, que contiene las variables de usuario, y otros datos necesarios para operar adecuadamente con ellas. Esta estructura tiene el siguiente aspecto:

```
typedef struct {  
    std::map<std::string, double> userVars; // maps a user variable  
        ↪ name to its current value  
    std::vector<Rule> rules; // rules that apply to the group of user  
        ↪ variables  
    std::map<std::string, double> mapGenoToPop; // variable that maps  
        ↪ a genotype to its population  
} UserVarsInfo;
```

Donde:

- **userVars** es un mapa cuyas claves son strings con los nombres de las variables de usuario definidas y los valores son números reales que definen el valor de las distintas variables.
- **rules** es un listado con las reglas que controlan la actualización de las variables, definidas como estructuras Rule definidas justo antes.
- **mapGenoToPop** es un mapa que contiene las poblaciones de todos los genotipos en un momento dado de la simulación, es necesario para poder operar con estos valores al actualizar las variables de usuario empleando las reglas.

Una vez tenemos estas estructuras definidas, procedemos a la elaboración de las distintas funciones que implementan la funcionalidad del TAD. Comenzando por algunas funciones muy triviales que crean, destruyen, comparan, imprimen e implementan alguna funcionalidad muy simple de estas estructuras:

- Función que rellena la estructura Rule:

```
Rule createRule(std::string id, std::string condition, std::
    ↪ string action);
```

- Función que rellena la estructura UserVarsInfo:

```
UserVarsInfo createUserVarsInfo(Rcpp::List Rules,
    Rcpp::List userVars,
    const fitnessEffectsAll& fitnessEffects,
    const std::vector<spParamsP>& popParams,
    std::vector<Genotype> Genotypes);
```

- Función que añade una variable de usuario al mapa de variables de usuario de UserVarsInfo:

```
UserVarsInfo addUserVar(UserVarsInfo uvif, std::string name,
    ↪ double value);
```

- Función que añade una regla al listado de reglas de UserVarsInfo:

```
UserVarsInfo addRule(UserVarsInfo uvif, Rule r);
```

- Función que compara dos reglas definidas por la estructura Rule:

```
int compareRules(Rule r1, Rule r2);
```

- Función que elimina una regla del listado de reglas de la estructura UserVarsInfo:

```
UserVarsInfo destroyRule(UserVarsInfo uvif, Rule r);
```

- Función que imprime por pantalla una regla definida por la estructura Rule:

```
void printRule(Rule r);
```

- Función que imprime por pantalla las variables de usuario y reglas contenidas en una variable del tipo UserVarsInfo:

```
void printUserVarsInfo(UserVarsInfo uvif);
```

- Función que comprueba que cada una de las subexpresiones separadas por “,” contenidas en el atributo Action de la estructura Rule están bien definidas, con la forma <userVar>= <expresión>:

```
bool isValid(std::string equation);
```

Por último definimos las funciones que implementan la funcionalidad principal del TAD, estas son **executeRules** y **parseAction** que, en conjunto, permiten la actualización de los valores de las variables de usuario siguiendo lo especificado en las reglas.

- **executeRules**:

```
void executeRules (UserVarsInfo& uvif,
                  double &currentTime,
                  const std::map<std::string, double>
                    ↪ birthMap,
                  const std::map<std::string, double>
                    ↪ deathMap,
                  const std::map<std::string, double>
                    ↪ mutationMap);
```

Esta es la función principal que implementa el algoritmo de actualización de las variables de usuario explicado en el apartado 3.2. Para ello hace uso de la librería `exprtk` para parsear la expresión booleana contenida en el campo **Condition** para determinar si una regla debe ejecutarse o no, y, en caso afirmativo llama la segunda función mencionada **parseAction** para ejecutar la modificación de las variables pertinentes. Los parámetros que recibe esta función son:

- **uvif**: estructura `UserVarsInfo` (explicada al inicio de este punto) que contiene el listado de variables de usuario y reglas definidas para la simulación en ejecución. Se emplea para acceder a las reglas y variables de usuario, así como para añadir las variables de usuario a la tabla de símbolos de la librería `exprtk` para poder emplearlas en el parseo de las expresiones contenidas en las reglas. Contiene además también el campo **mapGenoToPop** con las poblaciones asociadas a cada genotipo, que se deben añadir también a la tabla de símbolos (corresponden a las variables `n_x`) y son necesarias para calcular la población total `N`.
- **currentTime**: tiempo actual de la simulación en el ciclo en que se está ejecutando la función. Se emplea para añadirlo como variable a la tabla de símbolos de `exprtk` para emplearse en el parseo de expresiones. Corresponde a la variable **T** explicada en el punto 3.2.
- **birthMap**, **deathMap** y **mutationMap**: mapas que contiene las tasas de nacimiento, muerte y mutación de todos los genotipos presentes en el momento actual de la simulación. Se añaden como variables a la tabla de símbolos de `exprtk`. Corresponden a las variables **birthRate**, **deathRate** y **mutationrate** respectivamente explicadas en el punto 3.2.

Conociendo estos datos, la función opera de la siguiente manera:

Algorithm 1 `executeRules`

Rellenamos la tabla de símbolos de `exprtk` con las variables mencionadas en el apartado 3.2 para que la librería pueda operar con ellas al parsear la **Condition**.

for regla **in** `uvif.rules` **do**

Parseamos **regla.Condition** con `exprtk` obteniendo **true/false**

if `regla.Condition == true` **then**

Llamamos a **parseAction**

end if

end for

▪ **parseAction**:

```
void parseAction(UserVarsInfo& uvif ,
                Rule rule ,
                double currentTime ,
                const std::map<std::string , double>
                    & birthMap ,
                const std::map<std::string , double>
                    & deathMap ,
                const std::map<std::string , double>
                    & mutationMap);
```

Esta función se encarga de parsear el campo **Action** de la regla mediante `exprtk` y modificar el listado de variables de usuario definidas según lo indicado en este campo. En este caso los parámetros que recibe son muy similares a los de la función

anterior **executeRules**, pero añadiendo **rule**, que contiene la regla concreta que se va a ejecutar. Esta segunda función tiene la siguiente estructura:

Algorithm 2 executeRules

Rellenamos la tabla de símbolos de `exprtk` con las variables mencionadas en el apartado 3.2 para que la librería pueda operar con ellas al parsear la **Action**.

Dividimos la string **rule.action** en **tokens** separados por “,” (cada token define la actualización de una variable)

for each token **do**

 Separamos la string que define la asignación por el “=” obteniendo el **nombreVariable** y la **expresión**

 Parseamos la **expresión** con `exprtk` obteniendo el **valor** numérico al que equivale `uvif.userVars[nombreVariable] = valor`

end for

4.5. Integración en el flujo principal de código

Una vez definidas la funcionalidad de las variables de usuario y las reglas en el TAD anteriormente explicado, se procede a la integración de esta en el flujo principal de código de la simulación tumoral. Para ello se han incluido fragmentos de código que ejecutan las funciones de actualización de las variables de usuario dentro de la función **nr_innerBNB** del fichero **BNB_nr.cpp**, que es la que implementa el algoritmo BNB de Mahler explicado en el punto 2 (TODO).

La actualización de las variables de usuario se incluye dentro de esta función, de manera que se ejecute una vez por cada ciclo de muestreo de la simulación. Con el fin de poder simular la terapia adaptativa, es necesario poder emplear estas variables de usuario calculadas en la ejecución de las intervenciones, por tanto, esta actualización debe realizarse antes de que estas se ejecuten en cada ciclo. Finalmente se ha decidido realizar la actualización justo en el punto entre la actualización de las poblaciones de los genotipos según las tasas de nacimiento, muerte y mutación, y la ejecución de las intervenciones; de este modo las variables de usuario se actualizan de acuerdo a los datos temporalmente más cercanos posibles a la ejecución de las intervenciones, que es, por el momento, la principal funcionalidad para la que se van a emplear estas variables.

Se valoró la posibilidad de recalcular de nuevo las variables de usuario después de la ejecución de las intervenciones, pues estas pueden variar ciertos datos tales como la población, que afectarían al valor de estas variables. Sin embargo, esta opción se terminó descartando puesto que, al modificar las variables 2 veces por ciclo, se estaría aumentando notablemente el coste computacional del algoritmo, proporcionando además una información que, si bien es más precisa, no genera un beneficio suficientemente significativo como para justificar el coste.

Para realizar la actualización de las variables en el algoritmo de simulación principal mencionado, añadimos a este el siguiente fragmento de código:

```

std::map<std::string, double> birthMap = evalFVarsFitness(
    ⇨ fitnessEffects, Genotypes, popParams, "birth");
std::map<std::string, double> deathMap = evalFVarsFitness(
    ⇨ fitnessEffects, Genotypes, popParams, "death");
std::map<std::string, double> mutationMap = evalFVarsFitness(
    ⇨ fitnessEffects, Genotypes, popParams, "mutation");
if(rules_length > 0 && userVars_length > 0){
    uvif.mapGenoToPop = evalFVars(fitnessEffects, Genotypes,
        ⇨ popParams, true);
    executeRules(uvif, currentTime, birthMap, deathMap,
        ⇨ mutationMap);
    auxValues = {};
    for (auto &item : uvif.userVars){
        auxValues.push_back(item.second);
    }
    auxValues.push_back(currentTime);
    userVarValues.push_back(auxValues);
}

```

En este fragmento cabe destacar la creación y uso de la función **evalFVarsFitness** que permite acceder a las tasas de nacimiento, muerte y mutación de los distintos genotipos presentes en este momento de la simulación; que se deben pasar como parámetros a **executeRules** para poder añadirse a la tabla de símbolos de `exprtk`. Vemos que de forma similar se emplea también la función **evalFVars**, que ya existía con anterioridad, para obtener los valores de las poblaciones de los distintos genotipos, valores que también se deben pasar como parámetro a **executeRules**. Una vez obtenidos todos estos valores necesarios, se procede a la llamada a **executeRules** para actualizar las variables, almacenando los valores de las variables resultantes en vectores auxiliares para posteriormente poder devolverlos a la salida.

Además de estas adiciones al algoritmo, se ha modificado también su estructura de salida para que incluya una lista con los nombres de las variables de usuario creadas durante la simulación, y una matriz con los valores que toman las variables en cada unidad de tiempo de la simulación (cada fila es una lista con los valores de las distintas variables para una unidad de tiempo, en el mismo orden en que aparecen en la lista con los nombres de las variables), esta matriz se rellena con los valores que se han ido almacenando en el vector auxiliar mencionado anteriormente.

Por último, para finalizar esta integración en el flujo principal de código, se han modificado los parámetros de entrada de todas las funciones en R de `OncoSimulR` para que reciban de forma adecuada los parámetros **userVars** y **rules**, y se han actualizado los ficheros `RcppExports.R`, `RcppExports.cpp` y `OncoSimulR_init.c`, para que estos mismos parámetros se pasen a C++ de forma adecuada.

4.6. Variables de usuario en las intervenciones. Terapia adaptativa

Como hemos visto en la sección 3.1, la finalidad principal de la creación de las variables de usuario es lograr la simulación de la terapia adaptativa. Para ello, ahora que contamos con la funcionalidad completa de las variables de usuario, se debe lograr que estas puedan emplearse para definir las expresiones que determinan las acciones a realizar en una intervención.

Por tanto, se ha incluido estas variables en la funcionalidad de las intervenciones. Para ello se han modificado las funciones **executeInterventions** y **parseWhatHappens** del fichero **interventions.cpp**, que controlan la ejecución de las intervenciones en la simulación. Ambas funciones reciben unas modificaciones muy similares que consisten en:

1. Modificamos ambas funciones para que para que reciban como argumento la estructura `userVarsInfo` con los datos de las variables de usuario.
2. Añadimos en ambas un pequeño fragmento de código para introducir las variables de usuario y sus valores a la tabla de símbolos de `exprtk` permitiendo así que se puedan emplear en el parseo de expresiones de las intervenciones.
3. Modificamos las cabeceras y llamadas a estas 2 funciones de forma adecuada para que incluyan `userVarsInfo` en todas las llamadas.

Con estos cambios se finaliza el desarrollo de la funcionalidad y se procede a las pruebas para comprobar su correcto funcionamiento.

5

Pruebas y Resultados

5.1. Comprobación de funcionalidad previa

- Comprobamos que la funcionalidad previa no se ha perdido ejecutando la batería de tests ya existentes para todo oncoSimulR con la herramienta testthat y tambien los manuales.
- Tras realizar el merge entre las ramas anteriores se detectaron una serie de fallos de la funcionalidad previa en los tests que fue necesario solucionar para proceder a la implementación de la nueva funcionalidad. Estos fallos estaban localizados en los tests de la funcionalidad de intervenciones que se ha aplicado anteriormente (o bien en estado del arte o en la sección de unificación de ramas).
- Para solucionar los fallos:
 - Añadir InterventionTimes a x\$other\$interventionTimes con los tiempos de las intervenciones para poder realizar tests (se rellena en BNB_nr.cpp tras haber cambiado executeInterventions)
 - Adelantar la ejecución de intervenciones para que se realice antes de rellenar las estructuras de salida para evitar que las actualizaciones de poblacion de las intervenciones se realicen 1 sampleEvery despues de lo debido y para que la tabla de poblaciones y la de birth y death rates estén correctamente actualizadas y sincronizadas entre sí
 - Comentar la comprobación de totPopSize != totalPop en reduceTotalPopulation (interventions.cpp) porque al haberlo adelantado totPopSize no está actualizado

- Añadir código para eliminar los genotipos con 0 población despues de realizar intervenciones para evitar el error que se obtenía: Unrecoverable exception: ti: popSize <= 0. spP.popSize = 0.000000. Aborting.
 - Adaptar los tests de intervenciones a estos nuevos cambios y comprobar que todos los tests se pasan correctamente
- Tras esto ademas realizamos algunos cambios en el código pues durante estas modificaciones se vió que en numerosas ocasiones se ejecutaban fragmentos de código innecesarios, que en una ejecución de una simulación de gran tamaño suponían un gran aumento del tiempo de ejecución, para ello:
- Modificamos la funcion `ExecuteInterventions` de `interventions.cpp`:
 - Hacer que devuelva `true` si hay intervencion y `false` si no y en caso de error solo lance excepción (esto permite el siguiente punto y permite obtener `InterventionTimes` desde fuera)
 - Hacer que solo se llame a `UpdatePopulations` si ha habido intervención (antes se ejecutaba en todas las iteraciones aunque no hubiese intervenciones)
 - Cambiar `ParseWhatHappens`, ahora devuelve `void` y si hay error solo lanza excepción

¿Optimización?

5.2. Tests de variables de usuario

- Creamos el fichero de tests para testthat `test.user_var.R` que contiene 8 tests para probar el correcto funcionamiento de las variables de usuario
- Probar que las variables se crean correctamente
 - Comprobar que 2 variables no pueden tener el mismo nombre
 - Comprobar que las reglas se crean correctamente
 - Comprobar que 2 reglas no pueden tener el mismo ID
 - Comprobar que la ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de la variable T)
 - Comprobar que la ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de la variable N)
 - Comprobar que la ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de las poblaciones por genotipo `n_x`)
 - Comprobar que la ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de la otras variables de usuario)

5.3. Tests de intervenciones

- Modificamos el fichero de tests de intervenciones `test-interventions.R` para incluir dos tests que compruebe que las intervenciones se ejecutan de forma adecuada cuando el trigger depende de variables de usuario y cuando la función de modificación emplea variables de usuario.

5.4. Resultados

- ¿Optmización?
- ¿?

6

Conclusiones y trabajo futuro

6.1. Conclusiones

6.2. Líneas de trabajo futuro

- Emplear variables de usuario en otras funcionalidades más allá de las intervenciones
- Permitir graficar variables de usuario en función del tiempo como una salida interesante de forma similar a la evolución de la población de los genotipos

Apéndices



Ejemplos de bloques y comandos útiles en LaTeX

A.1. Ejemplo de sección

La DARPA creó el protocolo de Internet [**ipv4sta**].

Citamos el acrónimo Field-Programmable Gate Array (FPGA).

Bitstream es una secuencia de bits.

La figura A.1 se utiliza en la portada.



Figura A.1: Logo de la Universidad Autónoma de madrid.

Código A.1: Algoritmo de ordenación Quicksort

```
#include <stdio.h>

void quick_sort (int *a, int n) {
    int i, j, p, t;
    if (n < 2)
        return;
    p = a[n / 2];
    for (i = 0, j = n - 1;; i++, j--) {
        while (a[i] < p)
            i++;
        while (p < a[j])
            j--;
        if (i >= j)
            break;
        t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
    quick_sort(a, i);
    quick_sort(a + i, n - i);
}
```

```
#include <stdio.h>

void quick_sort (int *a, int n) {
    int i, j, p, t;
    if (n < 2)
        return;
    p = a[n / 2];
    for (i = 0, j = n - 1;; i++, j--) {
        while (a[i] < p)
            i++;
        while (p < a[j])
            j--;
        if (i >= j)
            break;
        t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
    quick_sort(a, i);
    quick_sort(a + i, n - i);
}
```

La ecuación de Euler ($e^{\pm i\theta} = \cos \theta \pm i \sin \theta$) es citada frecuentemente como un ejemplo de belleza matemática.

$$a^2 + b^2 = c^2 \tag{A.1}$$